면접 학습 정리

> 목적: 이 프로젝트를 하면서 실제로 학습한 기술과, 면접에서 설명 가능한 수준의 이해를 한 곳에 모은 문서;

1. 이 프로젝트로 무엇을 학습했는가

이 프로젝트는 단순히 결과물을 만드는 작업이 아니라, 아래 내용을 실전으로 익히는 과정이었다.;

- `React` 와 `Next.js App Router` 기반의 화면/데이터 흐름;
- `Server Component` 와 `Client Component` 의 역할 분리;
- `Server Actions` 기반의 서버 중심 mutation 처리;
- `revalidatePath`, `router.refresh`, `cache()` 를 활용한 캐시/갱신 흐름;
- `Supabase + PostgreSQL + Drizzle` 조합의 관계형 데이터 모델링;
- `Better Auth` 기반 인증과 세션 처리;
- `Capacitor` 를 이용한 웹 코드베이스의 앱 확장;
- 자연어/이미지 입력을 `LLM` 파싱 흐름에 연결하는 방식;
- AI를 구현 도구로 쓰더라도 설계, 검증, 책임은 사람이 가져가야 한다는 점;

---

2. 프로젝트를 통해 이해한 핵심 개념

2-1. App Router

- Next.js의 파일 기반 라우팅 체계이며, 기본적으로 서버 중심 렌더링 흐름을 가져가기 좋다.;
- 이 프로젝트에서는 로그인 이후 사용하는 앱형 서비스라 서버에서 바로 데이터를 읽고 화면을 구성하는 방식이 잘 맞았다.;
- 공개 콘텐츠 사이트보다, 인증 후 데이터를 보여주는 서비스에 더 자연스럽다고 느꼈다.;

2-2. Server Component vs Client Component

- `Server Component` 는 서버에서 실행되고, DB 조회나 인증 기반 데이터 구성에 유리하다.;
- `Client Component` 는 브라우저에서 실행되며, 입력, 모달, 탭 전환, 상태 업데이트 같은 인터랙션에 사용한다.;
- 이 프로젝트에서는 조회와 초기 렌더는 서버에 두고, 입력과 UI 상태는 클라이언트에 두는 분리가 중요했다.;

2-3. Server Actions

- 서버에서 실행되는 함수를 클라이언트에서 일반 함수처럼 연결해 쓰는 방식이다.;
- 이 프로젝트에서는 별도의 REST API를 많이 나누기보다, 거래 생성/수정/삭제 같은 mutation을 `Server Actions` 로 처리했다.;
- 장점은 인증, 권한, DB 접근 로직을 서버 경계 안에 두기 쉽다는 점이었다.;
- 단점은 외부 클라이언트 확장이나 API 계약 분리가 필요한 상황에서는 별도 API 계층보다 덜 명확할 수 있다는 점이다.;

2-4. revalidatePath / router.refresh / cache

- `revalidatePath()` 는 특정 경로의 서버 캐시를 무효화하고 다시 계산하게 한다.;
- `router.refresh()` 는 현재 화면의 서버 데이터를 다시 받아오게 한다.;
- `cache()` 는 같은 서버 렌더 안에서 중복 호출 비용을 줄이는 데 유용했다.;
- 이 세 개를 통해 `조회는 서버`, `수정은 액션`, `갱신은 재검증` 구조를 이해했다.;

2-5. Suspense / loading.tsx

- 모든 화면을 한 번에 기다리기보다, 준비된 영역부터 보여주는 방식이 체감 성능에 중요하다는 점을 배웠다.;
- `loading.tsx`, 섹션별 `Suspense fallback`, 병렬 데이터 조회가 빠른 전환 UX에 도움을 준다는 걸 이해했다.;

---

3. 데이터와 백엔드 관점에서 배운 점

3-1. 왜 관계형 DB가 맞았는가

이 프로젝트의 핵심 도메인은 아래처럼 관계와 집계가 중요했다.;

- 거래;
- 카테고리;
- 자산/부채;
- 예산;
- 통계;

그래서 document 중심 저장소보다 `PostgreSQL` 같은 관계형 DB가 더 자연스럽다고 느꼈다.;

3-2. Supabase를 쓰며 배운 점

- DB 운영 부담을 줄인 상태에서 빠르게 시작할 수 있었다.;
- 입문자 입장에서 자료와 레퍼런스가 많아 학습 경로가 명확했다.;
- 반면 인프라를 전부 직접 운영한 경험이라고 말하기는 어렵다.;

3-3. Drizzle을 쓰며 배운 점

- 스키마와 TypeScript 타입을 함께 가져가는 것이 큰 장점이었다.;
- raw SQL보다 안전하고 읽기 쉬운 Query Builder 흐름이 좋았다.;
- 반면 스키마 변경과 마이그레이션은 프론트만 하던 때보다 더 조심스럽게 접근해야 했다.;

3-4. 왜 NoSQL보다 관계형 DB가 맞았는가

- 이 프로젝트는 실시간 동기화보다 거래, 카테고리, 계좌, 예산 간 참조 관계와 집계가 더 중요했다.;
- `SUM`, `GROUP BY`, 월별 집계, 카테고리별 집계, FK 기반 무결성이 핵심이라 스키마리스 저장소보다 관계형 모델이 더 적합했다.;
- 그래서 `Firebase` 같은 선택지가 불가능했다기보다, 이 도메인에서는 우선순위가 더 낮았다고 이해하게 됐다.;

3-5. 왜 Drizzle이 당시 상황에 맞았는가

- `Prisma` 는 생태계와 도구가 강하지만, 별도 스키마와 generate 흐름이 있고 서버리스 환경에선 번들 부담도 고려해야 했다.;
- 당시 나에게는 TypeScript 코드 안에서 스키마와 타입을 함께 다루는 `Drizzle` 이 더 단순하고 직관적이었다.;
- 즉 더 유명한 도구보다, 현재 프로젝트의 학습 비용과 구조 단순성에 맞는 도구를 고르는 판단을 해보게 됐다.;

---

4. 인증과 보안에서 배운 점

4-1. Better Auth

- 직접 인증을 구현하는 것보다 검증된 솔루션을 쓰는 편이 초기 위험을 줄인다고 판단했다.;
- 로그인/회원가입, 세션 쿠키, 보호 라우트, 인증 기반 리디렉션 흐름을 더 현실적으로 이해하게 됐다.;
- 기존 DB와 인증 흐름을 더 자연스럽게 붙일 수 있다는 점도 배웠다.;
- 가입 직후 기본 카테고리를 자동 시딩하거나, 세션 저장 전에 IP와 UserAgent를 최소화하는 훅을 인증 흐름 안에 넣을 수 있었다.;

4-1-1. 왜 Better Auth가 이 프로젝트에 맞았는가

- `NextAuth` 는 레퍼런스와 생태계가 강하지만, 이 프로젝트에서는 DB 훅과 Drizzle 기반 통합이 더 중요했다.;
- `Better Auth` 는 인증 자체보다, 인증 전후의 프로젝트 맞춤 로직을 붙이기에 더 유연하다고 느꼈다.;
- 즉 업계 표준보다 현재 구조와 도메인에 맞는 도구를 고르는 판단이 필요하다는 점을 배웠다.;

4-2. 보안 기본값

이 프로젝트를 통해 특히 아래 원칙을 명확히 배웠다.;

- 인증은 클라이언트가 아니라 서버에서 검증해야 한다.;
- 모든 데이터 조회/수정은 `userId` 범위를 명시적으로 확인해야 한다.;
- 공개 입력 경로는 Origin, 크기 제한, rate limit 같은 상한이 먼저 필요하다.;
- AI 입력 기능일수록 비정상 payload와 남용 방어가 중요하다.;

4-3. 입력 검증은 단순 길이 체크로 끝나지 않았다

- 텍스트 입력 필드에 이미지 `base64` 를 직접 붙여넣어 토큰을 대량 소모시키는 형태를 막기 위해 `base64` 위장 텍스트 탐지가 필요했다.;
- 이미지는 MIME 타입, 원본 바이트 크기, `base64` 길이, 실제 디코딩 가능 여부까지 다중 검증해야 더 안전하다는 점을 배웠다.;
- 클라이언트가 보내는 길이 정보는 자기 신고 값이기 때문에, 서버가 독립적으로 다시 검증해야 한다는 점도 중요했다.;

4-4. HMAC 해싱과 개인정보 최소화

- IP, UserAgent 같은 값은 원문 저장보다 최소화가 우선이라는 점을 배웠다.;
- 단순 해시보다 `HMAC` 기반 해싱이 더 적절한 이유도 이해하게 됐다.;
- 이유는 비밀키 없는 해시는 역추적 가능성이 더 크지만, `HMAC` 은 서버만 아는 키를 사용해 원문 추정을 어렵게 만들기 때문이다.;

4-5. IPv6는 개별 주소보다 대역 기준 사고가 필요했다

- IPv6는 주소가 자주 바뀔 수 있어 개별 주소 기준 rate limit이 실효성이 떨어질 수 있다.;
- 그래서 /64 같은 서브넷 단위로 묶어 봐야 의미 있는 제한이 가능하다는 점을 배웠다.;

---

5. 상태 관리에서 배운 점

5-1. 왜 Zustand를 안 썼는가

- 구현해보니 대부분 상태가 폼 입력, 시트 열림 여부, 필터 같은 로컬 UI 상태였다.;
- 전역 상태가 크지 않으면 상태 관리 계층을 하나 더 넣는 것이 오히려 복잡도를 늘릴 수 있다는 걸 배웠다.;

5-2. 왜 React Query를 안 썼는가

- 이미 `Server Component + Server Actions + revalidatePath + router.refresh` 로 서버 중심 흐름이 잡혀 있었다.;
- 여기에 클라이언트 캐시를 추가하면 현재 규모에서는 캐시 계층 이중화 가능성이 컸다.;
- 대신 background refetch, optimistic update, stale-while-revalidate UX는 상대적으로 약해질 수 있다는 트레이드오프도 함께 이해하게 됐다.;

---

6. AI 파싱 흐름에서 배운 점

6-1. 파싱 파이프라인은 단순 호출이 아니었다

실제 흐름은 대략 아래 순서로 이해할 수 있었다.;

1. 빈 입력 검증;
2. OOD 필터;
3. 은행 메시지 전처리;
4. provider 결정;
5. 카테고리/계좌 DB 조회;
6. 프롬프트 조립;
7. `LLM` 호출;
8. 응답 검증과 JSON 파싱;
9. 확인 시트 표시;
10. 확정 후 저장과 잔액 반영;

핵심은 `LLM 호출` 자체보다, 그 앞뒤의 검증과 사용자 확인 단계를 어떻게 두느냐였다.;

6-2. OOD 필터에서 배운 점

- 완벽한 차단보다 `정상 입력을 놓치지 않는 것`이 더 중요했다.;
- 애매하면 통과시키고, 명백히 가계부와 무관한 입력만 먼저 차단하는 편이 UX상 더 안전했다.;
- 잘못 들어온 거래 1건은 사용자가 확인 시트에서 지울 수 있지만, 정상 입력이 차단되면 사용자는 앱 자체를 불신할 수 있기 때문이다.;

6-3. Provider 라우팅에서 배운 점

- 하나의 `LLM`만 붙이는 것보다, 입력 길이와 이미지 여부, 장애 상황에 따라 provider를 나누는 전략이 가능하다는 걸 배웠다.;
- 중요한 건 벤더를 여러 개 쓰는 것 자체가 아니라, 인터페이스를 통일하고 폴백 기준을 명확히 두는 것이었다.;
- 이 경험을 통해 `OpenAI SDK 호환 인터페이스 + baseURL 전환`처럼 확장 가능한 구조의 중요성을 이해하게 됐다.;

6-4. 모든 실패를 폴백하면 안 된다

- provider 장애, 네트워크 오류, 타임아웃 같은 `복구 가능한 실패`만 다른 provider로 넘기는 게 맞다는 점을 배웠다.;
- OOD나 잘못된 입력처럼 콘텐츠 자체 문제는 다른 provider로 보내도 같은 실패일 가능성이 높아 비용 낭비가 된다.;

6-5. 이미지 provider 사용량과 쿨다운

- 이미지 provider 사용량은 보안 rate limit과 다르게 비용 최적화용 상태로 볼 수 있다는 점을 배웠다.;
- 이 경우엔 반드시 DB에 영속화할 필요가 없고, 인메모리 추적도 현실적인 선택이 될 수 있다.;
- 대신 메모리 정리 기준과 장애 시 쿨다운 정책이 함께 있어야 한다.;

6-6. 타임아웃도 입력 복잡도에 맞게 달라질 수 있다

- 짧은 입력과 긴 입력에 같은 타임아웃을 주면 UX가 어색해질 수 있다.;
- 입력 길이와 복잡도에 따라 기대 응답 시간을 다르게 두는 것이 사용자 체감과 안정성 사이의 균형에 도움이 된다는 점을 배웠다.;

6-7. LLM 응답 파싱은 방어적으로 짜야 한다

- `LLM`은 JSON만 달라고 해도 코드블록, 설명문, 배열, `<think>` 같은 예외 형식을 섞어 반환할 수 있다.;
- 그래서 응답 파싱은 단순 `JSON.parse()`가 아니라, 정제, JSON 추출, 거부 응답 처리, 타입 검증이 이어지는 다단계 방어가 필요하다는 걸 배웠다.;

6-8. AbortSignal의 서버 측 의미

- 클라이언트 취소와 별개로, 서버도 `AbortSignal` 기반 타임아웃을 가져야 유령 응답과 무한 대기를 줄일 수 있다.;
- 즉 사용자가 취소하지 않아도 서버가 오래 걸리는 `LLM` 호출을 정리할 수 있어야 한다.;

---

7. 접근성, 시맨틱, SEO에서 배운 점

- 로그인 기반 앱형 서비스라 초기 우선순위를 인증, AI 파싱, 데이터 정합성, 모바일 UX에 뒀다.;
- 그 결과 웹표준, 시맨틱, 접근성, SEO는 상대적으로 약한 상태로 남았다.;
- 중요한 건 이걸 `몰라서 못 했다` 가 아니라 `우선순위를 뒤로 뒀다` 고 설명해야 한다는 점이다.;
- 다시 한다면 시맨틱 구조, 확대 허용, 포커스 관리, 메타데이터 보강을 더 초기에 설계해야 한다.;
- 특히 `userScalable: false` 같은 선택은 앱형 UX 의도는 있었지만 접근성 관점에서는 재검토 대상이라는 점을 분명히 배웠다.;

---

8. Capacitor를 통해 배운 점

- 하나의 웹 코드베이스로 앱 확장성을 검증하는 것은 초기 단계에서 현실적인 선택일 수 있다.;
- 하지만 WebView 기반 특성상 브라우저 친화성, 접근성, SEO와 긴장이 생길 수 있다.;
- 즉 `앱형 UX` 와 `웹 표준 품질` 은 때로 충돌하고, 이때 어떤 기준을 먼저 둘지 설명할 수 있어야 한다.;

---

9. AI 활용에서 배운 점

8-1. AI가 많이 작성해도 개발자 책임은 사라지지 않는다

- 이 프로젝트는 AI가 실제 구현에 크게 기여했지만, 구조 선택과 최종 책임은 사람이 가져가야 했다.;
- 중요한 건 손코딩 비율보다 아래를 설명할 수 있느냐였다.;
  - 왜 이 구조를 택했는가;
  - 어떤 기술을 넣고 뺐는가;
  - 어떤 위험을 감수했는가;
  - 무엇을 검증했고 무엇이 아직 약한가;

8-2. AI를 쓸수록 검증 루프가 더 중요하다

- 승인, 리뷰, QA, 보안 체크를 문서화한 이유도 여기에 있다.;
- AI는 속도를 높여주지만, 범위 일탈이나 과잉 구현을 만들기 쉬워서 오히려 더 명시적인 기준이 필요했다.;

---

10. 이 프로젝트에서 가장 큰 학습 성과

이 프로젝트의 가장 큰 성과는 `최신 스택을 많이 써봤다` 가 아니다.;

> 새로운 프레임워크를 배우는 과정에서도, 실제 사용자 문제를 정의하고, 설계와 구현과 검증을 연결해서, 서비스 흐름을 끝까지 완성해본 경험;

즉 아래를 한 사이클로 이해하게 된 것이 가장 컸다.;

- 문제 정의;
- 제품 목표 설정;
- 기술 선택;
- 데이터 모델 설계;
- 인증과 보안 고려;
- AI 기능 연결;
- 성능/UX 보완;
- 약점과 다음 개선 계획 정리;

---

11. 면접에서 이렇게 연결해서 말하면 좋다

- `이 프로젝트는 저한테 React/Next를 배우기 위한 튜토리얼이 아니라, 실제 문제를 해결하면서 구조적 판단을 훈련한 프로젝트였습니다.`
- `저는 이 프로젝트에서 무엇을 만들었는지보다, 왜 그 구조를 택했고 어떤 걸 일부러 넣지 않았는지 설명할 수 있게 됐습니다.`
- `특히 상태 관리, 서버 중심 데이터 흐름, 관계형 DB 모델링, 접근성 우선순위 같은 부분에서 트레이드오프를 더 명확히 이해하게 됐습니다.`